

基于 Termux 的多模型 QQ 群聊机器人系统

3099404236¹

¹独立开发者

本报告记录一套长期运行在 Redmi Android 手机上的 QQ 群聊 AI 机器人系统。系统以 NapCat 和 AstrBot 为接入与编排层，通过 OpenAI 兼容桥连接持久化 Antigravity 终端，并在用户明确指定时路由到 Opus、Exa MCP 或 ChatGPT 网页会话。与只描述最终架构不同，本版报告依据 2026 年 7 月 31 日至 8 月 1 日的两份私有工程记录、运行日志和手机实测，重建了从部署、登录、上下文复用到故障恢复的完整演进过程。报告重点分析九类真实故障：网页输入的二次复杂度与草稿滚雪球、Chrome 长对话内存增长、跨层超时失配、单通道并发阻塞、PTY 背压与静默思考误判、网页 DOM 结构漂移、图片工具循环、QQ 长回复节点上限，以及 Exa MCP 注册竞态和会话污染。最终实现将 39,142 字符全量上下文的发送阶段从 103.2 秒降到 2.58 秒，并以分层记忆、进度型超时、会话轮换、用户可见结果监控和可恢复守护脚本形成一套适合资源受限手机的工程方法。公开代码来自生产手机的只读快照；账号、Cookie、OAuth 状态、API Key、聊天数据库、日志、二维码、浏览器 Profile、WorkBuddy 原始记录和模型会话均未公开。

Termux | QQ bot | context engineering | multi-model routing | browser automation | reliability engineering

 [GitHub: 3099404236/termux-multimodel-qq-bot](https://github.com/3099404236/termux-multimodel-qq-bot)

证据	规模或范围	用途
生产手机只读快照	2026-08-01	确认公开源码与真实运行版本一致
WorkBuddy 工程记忆	2 份, 888 行	恢复故障现象、判断变化、修复与验证
私有会话事件	1,720 条	核对工具调用、时间线和操作前后关系
单元与语法检查	Python、Node、Shell	防止修复引入协议、格式和启动回归
手机现场测量	内存、耗时、HTTP、CDP	区分猜测、相关性和可验证根因

表 1 报告使用的证据类型。私有材料只用于分析，没有进入公开仓库。

研究对象与证据边界

项目的目标不是在手机上运行大语言模型本体，而是把闲置 Android 手机变成一台长期在线的轻量编排主机。QQ 消息进入后，手机负责接入、上下文整理、路由、网页或终端交互、工具调用和结果回传，真正的推理仍由外部模型完成。这样的部署可以替代临时没有的小主机，但也把移动系统后台限制、网页登录状态、交互式终端和网页 DOM 变化同时带进了生产链路。

上一版报告只根据最终代码解释架构，因此只有 3 页，也丢掉了最有价值的排障过程。本版新增的主要证据来自两份 WorkBuddy 工程记忆，共 888 行，覆盖 2026 年 7 月 31 日和 8 月 1 日；另有 1,720 条私有会话事件用于交叉定位时间顺序。报告只抽取技术事实、性能数字和可复现机制，不发布原始记录。原始材料中含账号、群号、局域网地址、进程号、临时路径和登录状态，继续保留在本机并被 .gitignore 排除。

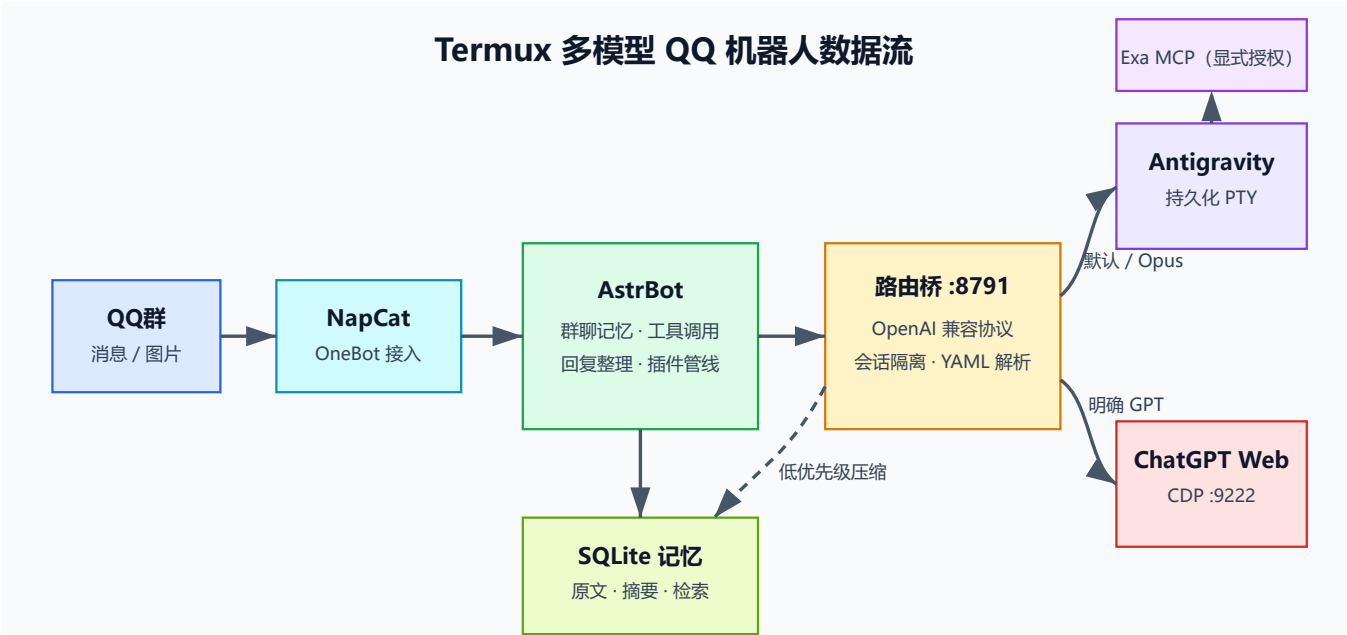
工程过程中曾多次出现“第一直觉不成立”的情况。例如最初把网页人格丢失归因于上下文没有传递，现场却证明是 ProseMirror 编辑器状态没有同步；最初把

Antigravity 卡死归因于对话过长，统计却显示成功与失败的平均消息数接近；最初把监控红灯等同于用户故障，后来发现大多数内部卡顿都被回退机制救回。因而本报告采用统一的事态分析结构：症状、证据、根因、修复、验证，并明确记录被排除的解释。

系统目标、约束与拓扑

系统需要同时满足六个约束。第一，手机可用内存有限，浏览器、QQ、AstrBot 和两个桥不能无限增长。第二，Antigravity 是全屏交互式终端，输入输出都存在缓冲和同步问题。第三，QQ、Google 和 ChatGPT 的扫码或网页登录不能被硬编码。第四，群聊里不同用户可能连续提问，后台记忆压缩不能阻塞前台回复。第五，长回答、图片和工具调用必须转换成 QQ 能稳定发送的消息类型。第六，USB 只应作为调试和首次配置通道，日常运行不能依赖电脑常开。

如图 1 所示，NapCat 负责 QQ 与 OneBot，AstrBot 负责事件管线、群聊记忆、工具执行和结果发送。路由桥监听 8791，默认请求进入 Antigravity 持久化 PTY；明确指定 GPT 的请求转到 8766 网页桥；明确指定 Opus 的请



所有服务默认仅监听 127.0.0.1；账号、Cookie、API Key 与聊天数据不进入公开仓库。

图 1 完整数据流。AstrBot 面向一个 OpenAI 兼容入口，桥接层再按显式意图拆分模型和会话。

组件	端口	职责
NapCat	6099	QQ 登录、OneBot 接入与下线状态
AstrBot	6185	事件、人格、记忆、工具、重试和发送
Antigravity 路由桥	8791	协议转换、模型选择、PTY 会话、YAML 解析
ChatGPT 网页桥	8766	裸 CDP、网页会话复用、文本和图片返回
Chrome CDP	9222	控制手机上已经登录的原生 Chrome 标签页
栈监控	8799	逐跳健康、近期失败和用户可见结果

表 2 本机服务与职责。除监控页面按需开放外，服务默认只监听回环地址。

求使用独立模型会话；明确授权 Exa 时，Antigravity 才能在当前轮调用 Exa MCP。模型路由与会话标识分离，因此同一群的默认、Opus 和 GPT 对话不会互相污染。

从无头浏览器到手机原生 Chrome

初始方案与失败边界。

Termux 原生 Chromium 149 可以在补齐 HOME、TMPDIR 并加入 --no-sandbox、--disable-gpu、--disable-dev-shm-usage 后启动，CDP 调试端口也能工作。现场测量显示空页约占 500 MiB，打开 ChatGPT 后约 550 MiB。这个结果证明“手机能跑浏览器”，但并不等于“能稳定登录并长期运行”。

无头浏览器连续三次卡在 Cloudflare 验证。移动 UA 还会触发通用错误，改成桌面 UA 和 1280 x 800 视口后页面可以渲染，但 Google OAuth 仍不稳定。DevTools screencast 能看到画面却不能可靠点击，最终需要自己写 CDP 遥控器，把电脑上的鼠标、键盘和截图转成 Input.dispatchMouseEvent 等协议事件。即便如此，无头指纹仍使登录链路不够稳健。

自连接 CDP 链路。

最终方案改用手机原生 Chrome 完成人工登录，再让 Termux 通过手机自己的 adbd 控制它。电脑只需首次执行 TCP 调试授权；随后 Termux 内的 adb 连接 127.0.0.1:5555，并把 tcp:9222 转发到 localabstract:chrome_devtools_remote。网页桥访问的始终是手机本机 9222，不经过 USB。

这条链路解决了 Cloudflare 和账号登录问题，也说明拔掉 USB 后机器人仍可工作。但是 service.adb.tcp.port=5555 并不持久，手机重启后可能恢复为仅 USB 模式。此时 Antigravity 路由仍然可用，ChatGPT 路由则需要重新打开 TCP 调试和 CDP forward。报告把它列为运维限制，而不是假装已经彻底自动化。

上下文、人格与群聊记忆

网页会话的首发全量与后续增量。

ChatGPT 网页不是无状态 API。新建网页对话时，桥必须把完整历史、系统人格、当前问题和输出协议一次性发送；同一个网页对话后续只发送系统控制消息和最新用户消息，让网页自身保存前文。工程记录中曾出现一次错误优化：为了降低首发体量截断历史，结果破坏了用户明确要求的上下文完整性。最终策略恢复为“新对话全量首发，旧对话增量复用”。

人格丢失经历了两轮修正。第一轮中，ClipboardEvent 让 DOM 看起来有文字，但 ProseMirror 内部状态没有同步，发送按钮不稳定，网页里只留下最后一段，因此完整人格没有真正送达。临时改用 execCommand("insertText") 后人格恢复。第二轮性能分析又发现，大文本 insertText 会导致编辑器反复处理整个文档。最终方案回到分块 paste，但每块控制为 4,000 字符，并加入发送就绪、残留草稿和页面复位检测。结论不是“paste 永远可靠”或“insertText 永远可靠”，而是必须同时验证编辑器状态、发送按钮和实际对话内容。

YAML 信封与用户可见文本。

两条模型链路使用同一逻辑信封：最终回答或工具调用必须被包装为 <<<AGY_RESPONSE>>> 到 <<<END_AGY_RESPONSE>>> 的 YAML 块。桥负责解析、清理和类型转换，AstrBot 只接收干净文本或工具调用。这样可以过滤终端里的思考状态、搜索进度、token 统计和网页装饰。

网页模型并不总是严格服从格式。一次真实回答只有开头标记，没有结尾标记，旧解析器匹配失败后把整个脚手架原样发进 QQ。修复后的解析器允许缺失尾标记、按公共缩进解析块标量，并在实在无法解析时至少剥离所有协议标记。这个案例说明结构化输出不能只靠提示词，接收端还必须防御残缺协议。

群聊长期记忆。

路线	登录成功率	常驻成本	最终判断
Termux 无头 Chromium	被 Cloudflare 反复拦截	约 500-550 MiB	只保留为诊断与备用
Playwright 接原生 Chrome	登录可复用	Node 驱动额外开销	Android 平台补丁和 CDP 握手不稳
裸 CDP 接原生 Chrome	复用真人登录	只有轻量 Node 服务	最终生产路线

表 3 浏览器路线的实机比较。

网页会话和 PTY 会话解决的是短中期连续对话，群聊历史则由 AstrBot 的 SQLite 记忆层负责。默认注入最近 18 条原文和最近 3 批摘要；每 30 条消息形成一批，群聊安静 90 秒后异步压缩。摘要固定保留话题、事实与决定、参与者立场、未解决问题、资源和检索关键词，避免自由摘要静默丢失“谁说了什么”。

模型还拥有历史检索工具。遇到“之前说过什么”、某个成员的观点或当前上下文不足时，可以主动搜索原文与摘要。压缩任务使用 group-memory-compressor：会话前缀和低优先级，路由器强制它走默认通道，不能抢占 Opus 或 GPT。

显式模型路由与并发隔离

路由器只读取最新用户消息的明确意图。用 GPT、ChatGPT、大小写变体和句首直呼可以进入网页桥；不要用 GPT、只是提到 GPT 等否定表达优先。Opus 与 Exa 使用相同的正负意图原则。普通“搜索一下”只允许内置搜索，只有明确点名 Exa 才开放 MCP。

早期实现把所有请求放进一个最大并发为 1 的 _PriorityGate，而 GPT 判断发生在拿到 gate 之后。结果是一个持续数分钟的网页请求独占唯一槽位，连完全不使用网页的普通请求也排队。修复是在进入 gate 之前调用同一套路由判定：GPT 请求跳过 Antigravity gate，交给网页桥自己的单飞队列；默认和 Opus 请求仍由 PTY 会话串行化。

线上并发验证中，慢 GPT 请求先开始，20.1 秒后提交普通请求；普通请求在总时间 38.8 秒时完成，只运行了 18.6 秒，而 GPT 在 231.6 秒时完成。两条请求的 gate 等待均为 0，证明路由隔离消除了跨模型头阻塞。

故障案例一：网页输入背压与草稿滚雪球

症状。

当上下文增大后，发送阶段从几十秒恶化到 103-370 秒。日志反复出现 CDP Runtime.evaluate 10 秒超时和重连，AstrBot 的外层请求先超时，网页桥里的僵尸请求却继续运行，后续请求全部堆积。现场输入框曾残留 23,806 个字符，连全选删除都需要超过 90 秒。

根因。

第一，分块 execCommand("insertText") 让 ProseMirror 每次重新处理当前文档，累计成本接近二次复杂度。

第二，ChatGPT 把草稿存进 localStorage["oai/apps/conversationDrafts"]，刷新和普通跳转不会清理。失败文本会在下一次发送时继续叠加。第三，CDP 超时后立即关闭 WebSocket 是负优化：页面仍可能执行旧插入，重连再试会重复写入。第四，旧 socket 的 onclose 会把新 socket 置空，形成重连竞态。第五，发送阶段没有检查客户端取消，AstrBot 超时后工作仍持续数分钟。

修复与验证。

最终版本使用 4,000 字符的 ClipboardEvent 分块。分块低于 9,000 字符，是因为 10,000 字符会被网页自动变成“粘贴文本附件”，不再作为编辑器内联文本。发送前若发现输入框、草稿或旧错误卡片脏污，桥不再对大 DOM 做全选删除，而是删除草稿键并调用浏览器进程级 Page.navigate。即使渲染线程卡死，这个动作仍可在约 7.3 秒内恢复干净页面。

CDP 执行超时不再自动拆 socket；连接建立改为单飞并带 generation 守卫；paste 分块超时后不原地重试，因为页面可能已经应用了那一块。整次发送最多重试两次，每次都先复位页面。发送队列和每个阶段都检查取消信号。最终 138 条消息、39,142 字符的全量 transcript 端到端约 24 秒，脏页面和中途取消场景也能在下一请求恢复。

故障案例二：Chrome 长对话内存增长

ChatGPT 是重型 React 应用。网页发送成功并不意味着旧消息会从 DOM 和组件状态中释放。长对话时，Chrome 渲染进程达到 404 MiB，系统空闲内存只剩约 65 MiB；CDP 往返开始出现 10-30 秒停顿，82 条消息的发送阶段达到 163 秒，总耗时 189 秒，超过当时 AstrBot 的 180 秒预算。

手动新建对话后，渲染进程从 404 MiB 降到 193 MiB，Chrome 总占用约 270 MiB；同一规模请求发送降到 14.5 秒，总耗时降到 28.6 秒。进一步把网页会话轮换从 24 轮改为 10 轮后，另一次现场测量中渲染进程从 417 MiB 降到 210 MiB，系统空闲内存从 65 MiB 回到 235 MiB，发送阶段约 1 秒。

因此最终采用两个轮换阈值：最多 10 轮或累计 180,000 字符。轮换并不删除长期上下文，因为新对话会重新首发完整 transcript；代价是每 10 轮出现一次较大的首发请求。这个设计把网页 DOM 设为有界缓存，而不是错误地把它当成永久记忆数据库。

输入规模	旧方案	分块 paste	观察
4,000 字符	23.6 s	0.371 s	单次 paste 只形成一个编辑器事务
12,000 字符	140.5 s	数秒级	旧方案随总文本快速恶化
35,000 字符	超过 600 s	6.5 s	旧方案未能完成
39,142 字符	103.2 s	2.58 s	最终生产版本的发送阶段

表 4 手机 Chrome 上的输入性能对比。不同批次页面状态不同，数字用于说明数量级。

层或阶段	旧值	最终值	原则
AstrBot provider	180 s	600 s	最外层必须最宽
GPT 路由中间层	180 s	570 s	先于外层清理
网页无进度	固定 180-300 s	120 s 无进展	有生成信号就继续
网页硬上限	300 s	1,800 s	只防无限等待
Antigravity 提交	5 s	20 s	容忍手机瞬时负载
Antigravity 无进度	60 s	180 s	容忍静默思考和搜索
Antigravity 总体	120-150 s	300 s	保留回退恢复时间

表 5 跨层超时预算。最短一跳决定用户实际等待上限。

故障案例三：最短超时与跨层取消

一次复杂搜索在网页上已经生成正文，桥却突然新开对话并重发 83,134 字符。原因是超时链有三层：AstrBot 600 秒、Antigravity 路由桥固定 180 秒、网页桥按进度等待。工程过程中只放宽了两端，漏掉中间层，所以真正的系统预算仍是最短的 180 秒。

路由桥超时后关闭 HTTP 客户端，网页桥收到取消并复位页面；AstrBot 再重试，就表现为“明明快答完了又开一个”。修复后网页路由中间层使用环境变量 CHATGPT_WEB_TIMEOUT_S，默认 570 秒，略低于 AstrBot 的 600 秒，让它有时间清理并返回可见错误。网页桥内部不再用固定总时长判断正常工作，而是只在连续 120 秒没有停止按钮、文字增长或新图片时判定无进展，30 分钟仅作为硬上限。

Antigravity 链路采用另一组相容预算：提交 20 秒、无进度 180 秒、总体和 print 回退 300 秒、AstrBot 外层 600 秒。成功请求平均 41 条消息，失败请求平均 39 条，且失败中存在只有 2 条消息的会话，因此“长对话导致 PTY 卡死”被排除；真正问题是手机负载下提交、静默思考和搜索超过旧固定阈值。

故障案例四：PTY 背压、状态同步与恢复

Antigravity TUI 会持续全屏重绘。如果桥只在请求期间读取输出，空闲刷新也可能塞满 PTY 输出缓冲；下一次同步写入又塞满输入缓冲，形成双向背压。持久会话最终采用专属工作线程持续排空输出，滚动缓冲保存应用层快照，其他请求只能排队，不能由多个线程同时写同一终端。

终端尺寸从高滚动区缩到 160 x 50，降低整屏刷新量；历史由应用缓冲保存，不依赖终端本身。写入、提交、生成进度分别检测，异常会关闭整条会话并等待线程退出，再用新进程重试。取消也必须等待清理完成，防止 `asyncio.to_thread()` 中的孤儿线程继续占用旧 PTY。

监控上线当天记录到 25 次内部卡顿，其中 19 次被持久终端恢复或 print 回退救回，只有 6 次真正造成失败。这个数字后来直接改变了监控设计：内部卡顿不能等同于用户故障，否则面板会天天报红，却无法描述群友是否收到答案。

故障案例五：网页结构漂移与输出清洗

网页桥最初直接读取 `.markdown.innerText`，随后暴露出四类结构丢失。第一，HTML 有序列表的编号来自 `CSS::marker`，不在 `innerText` 中，网页上的 1、2、3 到 QQ 后全部消失。第二，代码块 `<pre>` 里同时包含语言标签、复制按钮和代码，`textContent` 会把它们粘成 `Bashmvn test`。第三，引用角标是带 `data-testid` 的胶囊标签，序列化后变成正文中的孤立域名和 `+1`。第四，一个列表项内部的多个 `<p>` 被统一用双换行连接，造成大量无意义空行。

最终实现不依赖单个 `innerText`，而是遍历 DOM：有序列表显式生成编号，无序列表生成项目符号，嵌套层级保留两格缩进，代码正文只取 `pre > code`，表格按行序列化，标题降为普通文本。引用角标按用户选择完全隐藏。列表内部使用单换行，顶层块之间才使用空行；代码围栏由状态机保护，避免全局正则误删代码中的空行。

另一次网页更新把错误按钮的 `data-testid` 从旧值改成 `regenerate-thread-error-button`，错误文本本身只是裸 `<p>`，还位于 `.markdown` 中。旧桥既没有触发重试，又把“Something went wrong”当答案发入群。修复采取结构、按钮和中英文文本三重检测，并在每个返回出口调用 `rejectErrorCard()`。发送前若页面还残留上一轮错误卡片，必须先复位，否则会点错轮次。

这些案例形成一个通用原则：网页自动化选择器必须有结构或文本兜底；采集层必须把网页语义重新编码为稳定的中间格式，不能把浏览器可见文本误认为模型原始输出。

故障案例六：图片回答与 Agent 循环

纯图片回答没有 `.markdown`、没有文字，甚至没有稳定的消息角色容器，旧桥持续看到 0 字符，连续两次等待 120 秒后失败。修复后，桥把最后一轮中自然宽度不小于 200 像素的新图片也视为进展和完成。图片 URL 依赖登录 Cookie，必须在页面上下文内 `fetch`，再经 CDP 返回 `base64`；现场 3 MiB 图片约 0.8 秒下载。文件写入 `proot` 可见目录后，以 `send_message_to_user` 工具调用交给 AstrBot。

这个方案上线后立刻出现第二个问题：一次画图请求连续生成 5 张。AstrBot 执行工具后会把工具结果再次送给模型；无状态网页桥把它当新提问，又画一张并返回同一工具调用，形成 Agent 循环。修复是检查最后一条工具结果是否应答本桥刚返回的图片工具调用，若图片已经送达，就在 0.2 秒内返回空内容和 `finish_reason: stop`，不再访问页面。新的用户消息会越过这个终止条件，继续正常回答。

Antigravity 自己也有原生图片步骤，图片由运行时写入会话目录，再通过同一发送工具回传。提示词只描述“允许使用内置图像生成能力”，不写死内部步骤名。此前 Exa 故障证明，提示词写死一个模型看不到的工具名会诱导模型编造调用。

故障案例七：QQ 长回答节点上限

一次 30,981 字节长回答在桥端成功返回，QQ 却没有任何消息。AstrBot 当时把整条回答塞进一个合并转发节点；从历史真实流量反推，19,584 字节成功，而 30,981 字节失败。QQ 报错为转发节点为空，说明单节点存在实际长度边界。

修复在发送装饰阶段按 20,000 UTF-8 字节切分，优先在换行或空格处断开，绝不切坏多字节字符。多个片段必须放进一个 `Nodes` 容器，否则适配器会把每个 `Node` 分别发成一张转发卡片。55,300 字节测试被切成 19,987、19,987 和 15,326 字节三个节点，内容和图片均保留；短回答继续沿用单节点行为。`forward_threshold` 没有被擅自修改。

故障案例八：Exa MCP 注册竞态与会话污染

三次明确要求 Exa 的请求没有真正搜索。日志证明不是网络、API Key 或权限问题，而是 Antigravity CLI 在 OAuth 登录后并行初始化 MCP 和提交 `prompt`。正常时 Exa schema 比用户消息早 0.3-2.4 秒写入；失败时消息先转发，工具尚未进入工具表。手机高负载把 MCP 初始化从约 2 秒拖到 6.35 秒，使竞态更容易发生。

工具缺席后，模型根据提示词连续编造三个工具名，全部被 `unknown tool name` 拒绝。更严重的是持久会话记住了这些失败，第三次甚至不再尝试工具，8.6 秒内直接凭知

识回答。失败历史会持续到 20 轮轮换或 6 小时空闲，因此被称为“会话污染”。

临时缓解是停止桥、删除对应会话键并重启，使内存注册表和文件状态一致。随后三个隔离探针都真实调用了 Exa。竞态本身仍未完全解决，因为 `prompt` 何时提交由 CLI 内部决定；报告把它明确列为未解决风险，而不是用重试掩盖。诊断探针应使用独立 `user=diagnostic:...` 会话，避免污染群聊历史。

故障案例九：QQ 掉线与监控误判

NapCat 被服务器踢下线时，消息根本没有进入手机，但 AstrBot、两个桥和所有 HTTP 端口仍然健康。旧监控只看“进程和端口是否存在”，20 分钟内完全没有发现。快速登录又会因为不可用的缓存会话提示账号已经登录；只关闭 `screen` 也不会清理 `proot` 下的 QQ、`Xvfb` 进程树，新旧实例会争抢账号。

新增 `watchdog` 以 AstrBot 日志最后一条 QQ 入站消息的哈希为主信号，只有长时间没有变化时才做昂贵检查，并把“群里安静”和“确认离线”分开。自动重启默认关闭，因为安静群聊误判后重启 QQ 比原问题更严重。扫码登录、旧进程清理和冷却期均被写入脚本。

监控自身也经历三次修正。最初用“今日累计错误是否为零”控制灯色，导致当天只要出过一次故障就永远红。随后改为最近 15 分钟决定颜色，今日累计只做说明。第二次又把所有 PTY 卡顿算成失败，但当天 25 次卡顿有 19 次自动恢复。最终改为用户是否收到回复：桥活性、最近成功与失败、错误是否被 AstrBot 吞掉。数据口径也修复了 UTC 日期、无日期日志跨午夜和 ChatGPT 日志路径错误。

可靠性设计总览

九类故障最终收敛为五条横切原则。

第一，状态必须有唯一所有者。每个 PTY 会话由一个工作线程独占，网页桥由单飞队列独占页面，跨模型请求在路由前分流。第二，超时必须按完整调用链设计，并以进度而不是固定总时长判断复杂任务。第三，恢复必须先清理旧状态再重试；无论是网页草稿、PTY 子进程、NapCat 进程树还是会话注册表，都不能在未知状态

监控对象	最终信号	为什么
NapCat / QQ	进程、掉线标记、消息流	端口健康不能证明账号在线
AstrBot	进程和 6185	确认编排层存活
Antigravity 结果	近期是否有最终成功	内部卡顿可被回退救回
错误被吞	Suppressed runtime error reply	直接对应“用户提问但群里沉默”
ChatGPT 网页桥	8766、失败增量、出图	日志无时间戳，需要按新增字节统计
Chrome CDP	9222 HTTP	确认原生页面仍可控制
QQ 发送	近期发送失败	模型成功不等于消息送达

表 6 从内部实现指标转向用户可见结果的监控口径。

故障域	主要根因	核心修复	验证结果
网页输入	编辑器二次复杂度、草稿累积	4,000 字符 paste、O(1) 复位	103.2 s -> 2.58 s
网页内存	长对话 DOM 不释放	10 轮或 180k 字符轮换	404 -> 193 MiB
超时	最短中间层 180 s	600/570/进度型预算	长搜索不再被误切
并发	GPT 占用 Antigravity gate	路由前 bypass	普通请求不等 231.6 s
PTY	背压、过紧提交和进度阈值	持续读取、单线程、分阶段超时	19/25 次内部卡顿自愈
网页 DOM	选择器漂移、marker 丢失	语义序列化和文本兜底	列表、代码、错误卡片正常
图片	纯图片无文字、工具回合循环	图片进展检测和已送达终止	0.2 s 关闭工具回合
QQ 发送	单转发节点过长	20,000 字节安全拆分	55,300 字节无损三节点
Exa	MCP 注册竞态、失败历史污染	隔离探针和会话轮换	重置后三次真实调用

表 7 真实故障、修复和量化验证摘要。

层	运行环境	公开实现
QQ 接入	Termux/proot	deployment/termux/, NapCat 需外部安装
机器人核心	Ubuntu	仓库根目录、astrbot/, plugins/
默认模型桥	Ubuntu	bridges/antigravity/
GPT 网页桥	Termux	bridges/chatgpt-web/
报告与验证	电脑/CI	paper/, tests/, GitHub Actions

表 8 运行层与公开源码的对应关系。

上继续叠加。第四，监控应优先测量用户可见结果，而不是内部异常次数。第五，网页、CLI 和第三方服务都是不稳定边界，协议接收端必须容错，但不能把残缺控制文本泄漏给用户。

部署、守护与运维边界

系统由 Termux 外层和 proot Ubuntu 内层共同组成。NapCat、原生 Chrome、ChatGPT 网页桥和监控运行在 Termux 侧；AstrBot 与 Antigravity 路由桥运行在 Ubuntu 侧。启动脚本通过 proot 绑定 /dev、/proc，并使用 wake lock 降低休眠影响。

重启操作不能只看脚本名字。Antigravity 桥收到普通终止信号时可能被持久线程拖住，需要确认 PID 真正退出；从 Termux 直接执行 Ubuntu 内脚本会把 /root 解释成 Android 只读目录，必须通过 proot 启动。NapCat 的 screen 会话退出不代表 QQ 进程树退出。ChatGPT Node 服务有 watchdog，Antigravity 桥在快照中没有同等级守护，错误地杀掉它不会自动恢复。这些差异都写进 deployment/termux/ 的启动和监控脚本。

安全、隐私与公开快照

生产手机始终是本次发布的事实来源。本地旧仓库中有多个同名桥文件，其中一个只有几十行，是启动壳而不是实现；若仅凭文件名覆盖，可能把手机上的新版本回滚。因此导出时通过 run-as com.termux 和流式 tar 读取手机目录，排除数据目录、依赖、缓存和备份，再对关键桥文件计算哈希并同步到公开项目。手机上的运行代码和进程没有因发布而被修改或重启。

公开仓库不包含 QQ 号、群号、Cookie、OAuth code、API Key、二维码、浏览器 Profile、聊天数据库、日志、Antigravity 会话、WorkBuddy 记录或 PID 文件。两

个 NapCat 脚本中的账号默认值被替换为必须在手机本地提供的 NAPCAT_UIN。网页桥只监听回环地址；监控若需要局域网访问，应由用户显式开放。ChatGPT 网页自动化和第三方 CLI 仍受服务条款、页面更新和账号风控影响，正式生产环境应优先选择官方 API。

验证结果与可复现范围

公开快照完成了 Ruff 格式化与静态检查，桥接、路由和群记忆的 12 项目标测试通过；4 个独立 Python 文件可编译，2 个 Node 文件语法通过，34 个 Shell 脚本由 Termux Bash 校验。Typst PDF 在本地和 GitHub Actions 中构建。发布前又扫描了已知账号、设备标识、OAuth code、API Key、Google client id 和用户目录，未发现私有值。

这些验证证明的是协议、路由、切分、记忆和脚本结构在公开快照中一致，并不能替代完整的生产负载基准。性能数字来自同一台手机的真实故障前后测量，受页面状态、网络 and 模型生成时间影响，适合说明根因和数量级，不应被解读为所有设备上的固定吞吐量。

局限性与后续方向

第一，Exa MCP 初始化竞态仍位于第三方 CLI 内部，当前只能通过降载、会话轮换和隔离探针缓解。第二，手机重启后 TCP adbd 与 Chrome CDP 可能需要电脑重新授权，尚未做到真正无人值守恢复。第三，ChatGPT 网页 DOM、错误文案和 testid 会变化，语义序列化和多重兜底只能降低维护频率，不能消除风险。第四，厂商后台管理和低内存回收仍可能终止任一进程，wake lock 与任务锁定只是缓解。第五，显式模型指令容易理解和审计，但没有实现基于问题复杂度的自动路由。第六，结构化摘要仍会丢细节，需要继续用“压缩后答错”的真实案例更新模板和检索策略。

后续最值得优先处理的是三个方向：让 ChatGPT CDP 在重启后自动恢复；在 Exa 请求前获得可观察的 MCP ready 信号；建立可回放的故障测试集，覆盖草稿残留、错误卡片、工具循环、PTY 停滞、QQ 掉线和超长回复。这样项目才能从“能长期跑”进一步变成“每次修改都能证明没有退化”。

结论

这套系统的难点不在于把几个进程启动起来，而在于让多个不稳定边界在资源受限手机上共同工作。最终架构通过模型与会话分离、首发全量加增量复用、分层群聊记忆、持续读取 PTY、网页 O(1) 复位、进度型超时、路由前并发隔离、用户可见结果监控和安全的 QQ 消息拆分，把一系列“偶尔不回”的现象转化成可观测、可恢复、可测试的工程机制。

更重要的是，工程记录保留了错误判断如何被证据推翻：paste 与 insertText 的选择被两次修正，内部卡顿不再被等同于用户失败，超时不再只改最外层，工具调用不再假设 Agent 会自动终止。这些过程比最终参数本身更可复用，也正是本版报告补充 WorkBuddy 资料的原因。

项目代码：github.com/3099404236/termux-multimodel-qq-bot。